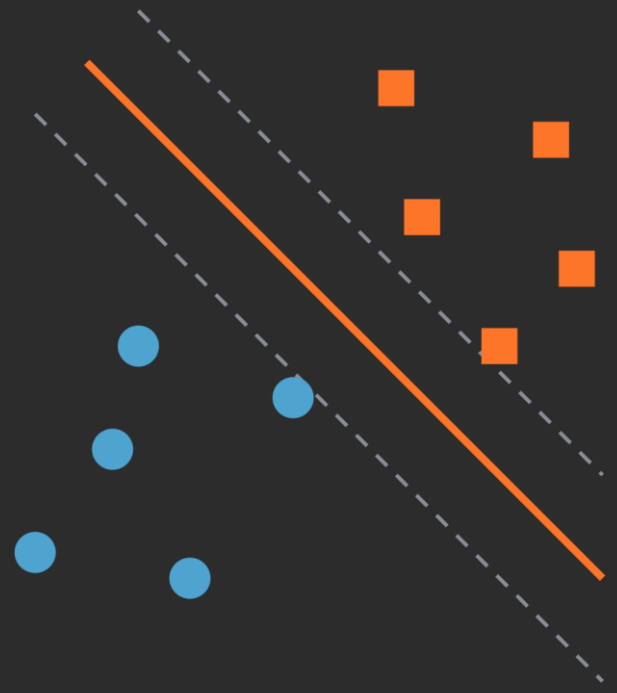
Practical machine learning

Support vector machines

Finding the widest street between two classes



Roadmap

Four ideas, one after another

01

Maximum margin

Pick the line with the widest street

02

Support vectors

Only the points on the edge matter

03

Soft margin and C

Allow a few mistakes, at a price

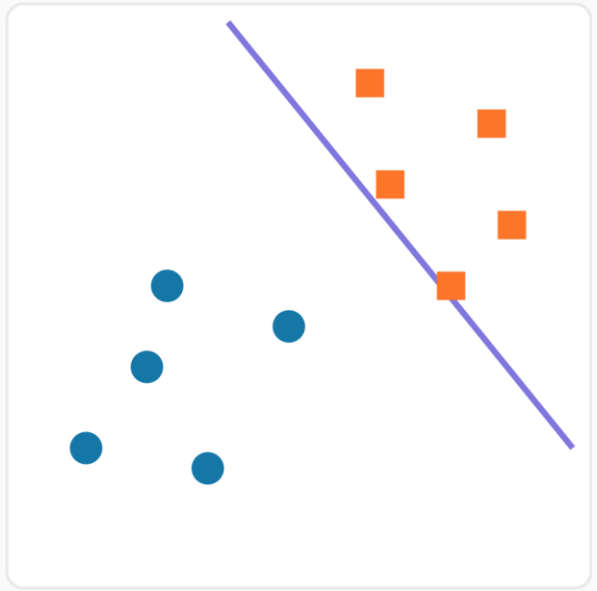
04

Kernels

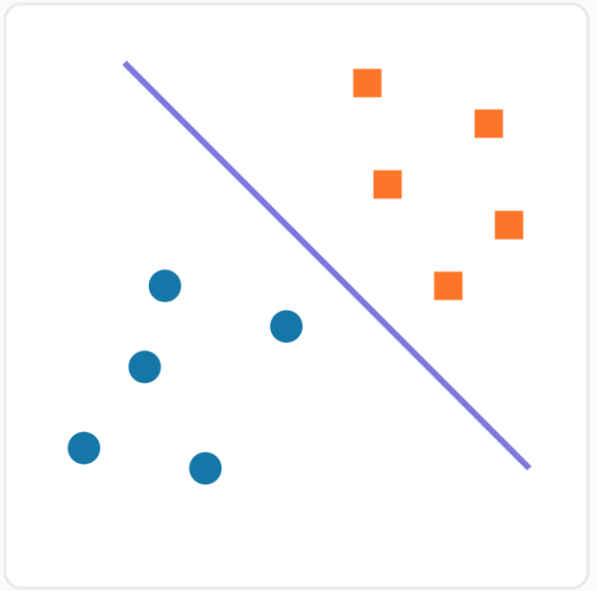
Bend the boundary when a line is not enough

Step 1 · Maximum margin

Both lines separate the data. Which one would you trust?

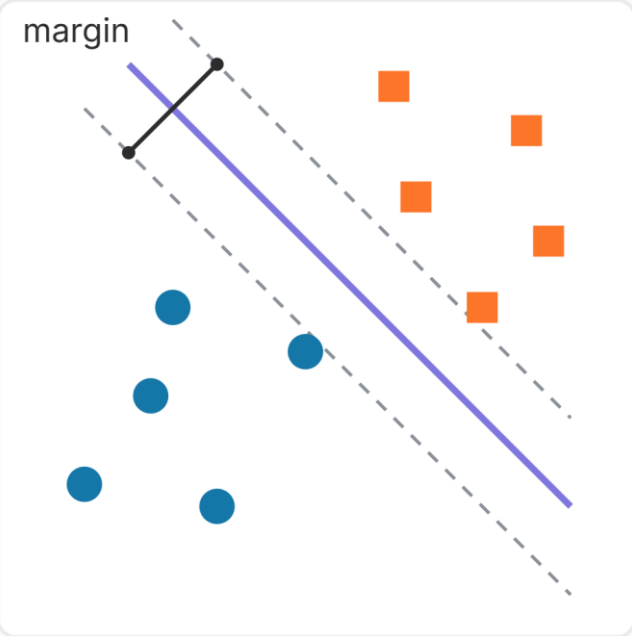


Line A



Line B

Choose the line with the widest gap



For perfectly separated data, the margin is the gap between the closest points of the two classes.

The dashed lines mark the edges of this margin, with the decision boundary in the middle.

SVM's goal: find the line with the maximum margin.

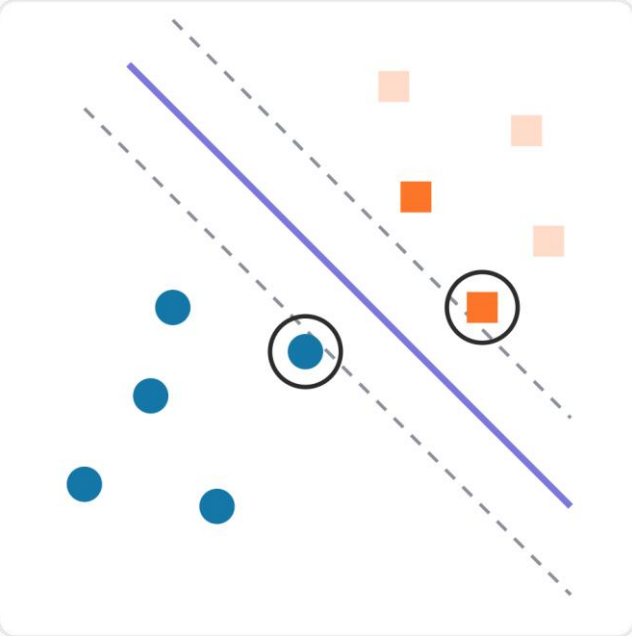
A wide margin often improves generalization, but it does not guarantee higher test accuracy.

In any number of dimensions, the boundary is a hyperplane

1 feature	2 features	3 features	n features
A single point	**A line**	**A flat plane**	**A hyperplane**

SVM finds the **maximum-margin hyperplane**: the widest street, in as many dimensions as your data has features.

Support vectors are the points that shape the boundary



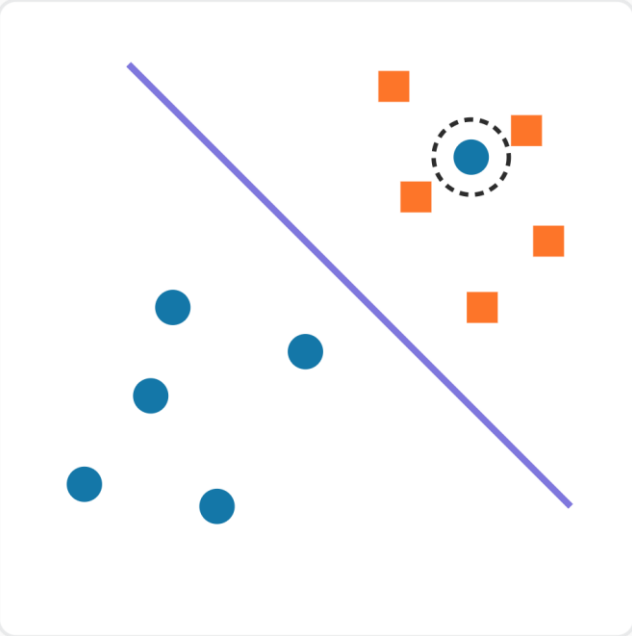
Here, the ringed points touch the margin edges. These are support vectors.

A point far from the margin usually has no effect on the boundary.

Moving a support vector can shift the separating boundary.

With a soft margin, points inside the margin can also be support vectors.

Real data is messy



One stray circle sits among the squares: perhaps noise, perhaps a labelling mistake.

Now no straight line separates the classes perfectly. Forcing a near-perfect split makes the street very narrow.

Discuss

Should SVM bend over backwards to classify this point, or accept it as a mistake to keep the wide street? What does each choice cost?

Soft margin: allow a few mistakes, at a price

Allow

Some points may sit inside the street, or even on the wrong side.

Penalize

Every violation adds a penalty to what SVM is minimizing.

Balance

SVM trades off a wide street against a small total penalty.

A hard margin demands a perfect split. A soft margin is the default in practice, because real data is rarely perfectly separable.

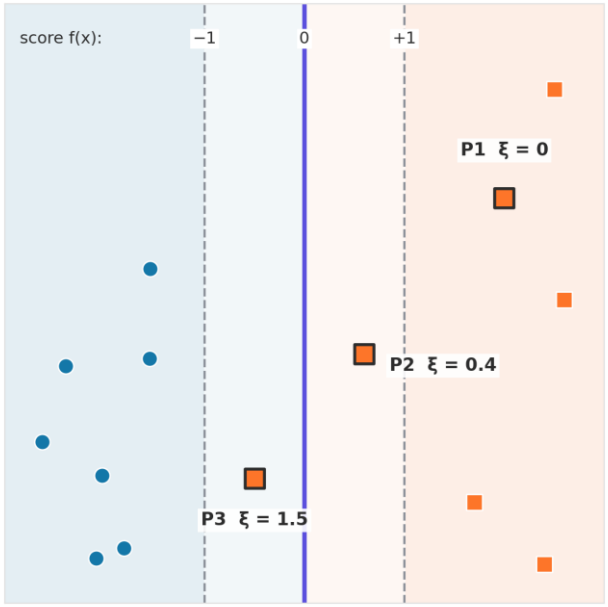
First, every point gets a score f(x)

$f(x) = w \cdot x + b = w_1 \times feature_1 + w_2 \times feature_2 + b$

Point ($feature_1$, $feature_2$)	Calculation with w = (1, 0.5), b = −4	Score f(x)	What SVM says
(5, 2)	1×5 + 0.5×2 − 4	2.0	Orange, outside the street
(4, 0.5)	1×4 + 0.5×0.5 − 4	0.25	Orange, inside the street
(2, 2)	1×2 + 0.5×2 − 4	−1.0	Blue, on the margin edge

w (weights) and b (bias) are learned by fit(). Positive: orange (+1). Negative: blue (−1). Street: −1 < f(x) < +1.

What counts as a mistake: the slack ξ



Each point gets a score f(x). The street runs from −1 to +1, with the boundary at 0.

P1: outside the street, on its own side. No penalty: ξ = 0.

P2: inside the street but still correct: 0 < ξ < 1. P3: on the wrong side: ξ > 1.

Slack ξ ("xi") = how far a point sits past its own margin edge.

How the slack ξ is calculated

$\xi = \max(0, 1 - y \cdot f(x))$ y = true class (+1 or −1)

Point (true class y = +1)	Score f(x)	$\xi = \max(0, 1 - y \cdot f(x))$	Where it sits
P1	2.0	max(0, 1 − 2.0) = 0	Outside: no penalty
P2	0.6	max(0, 1 − 0.6) = 0.4	Inside: small penalty
P3	−0.5	max(0, 1 + 0.5) = 1.5	Wrong side: misclassified

Total penalty = C × (0 + 0.4 + 1.5) = C × 1.9. With C = 0.1 that adds 0.19; with C = 10 it adds 19.

The same rule for blue points (y = −1)

$\xi = \max(0, 1 - y \cdot f(x))$ y = true class (+1 or −1)

Point (true class y = −1)	f(x), then y · f(x)	$\xi = \max(0, 1 - y \cdot f(x))$	Where it sits
B1	−2.0 → 2.0	max(0, 1 − 2.0) = 0	Outside: no penalty
B2	−0.6 → 0.6	max(0, 1 − 0.6) = 0.4	Inside: small penalty
B3	+0.5 → −0.5	max(0, 1 + 0.5) = 1.5	Wrong side: misclassified

Multiplying by y = −1 flips the sign. Then, for both classes, a bigger y · f(x) means more safely correct.

Step 3 · Soft margin

‖w‖, the size of the weights, sets the street width

‖w‖ = √(w₁² + w₂² + …) Margin width = 2 ÷ ‖w‖

One-feature model	Weight w	Street edges (f = −1 and +1)	Street width = 2 ÷ w
f(x) = 0.5x − 1	0.5	x = 0 and x = 4	4
f(x) = x − 2	1	x = 1 and x = 3	2
f(x) = 2x − 4	2	x = 1.5 and x = 2.5	1

All three have the boundary at x = 2. Smaller weights mean a wider street, so SVM tries to keep ‖w‖ small.

Step 3 · Soft margin

SVM minimizes ½‖w‖² + C × Σξᵢ

½‖w‖²: wide street

Small weights = wide street. The ½ and ² just tidy the maths.

Σξᵢ: few mistakes

Σ means "add up". Σξᵢ is the slack of every training point, added together.

C: the balance

Large C: mistakes cost more. Small C: a wide street matters more.

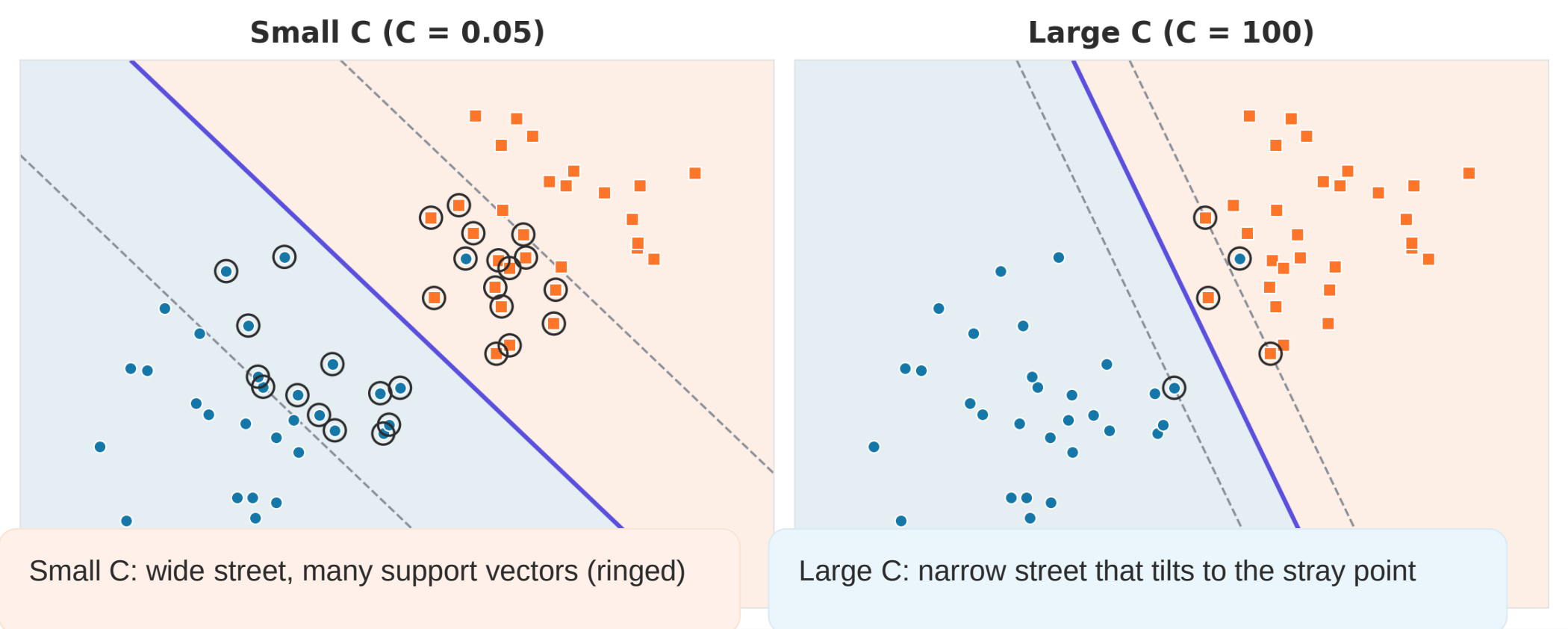
SVM does not try lines one by one. An optimizer finds the w and b with the lowest total cost; in scikit-learn this happens inside fit().

C sets the price of each mistake

	Small C	Large C
Each mistake is	Cheap	Expensive
The street is	Typically wider	Typically narrower
Training errors	More violations tolerated	Fewer violations preferred
Main risk	May underfit (high bias)	May overfit (high variance)

In scikit-learn, **SVC(C=1.0)** is the default. Tune C with cross-validation rather than guessing.

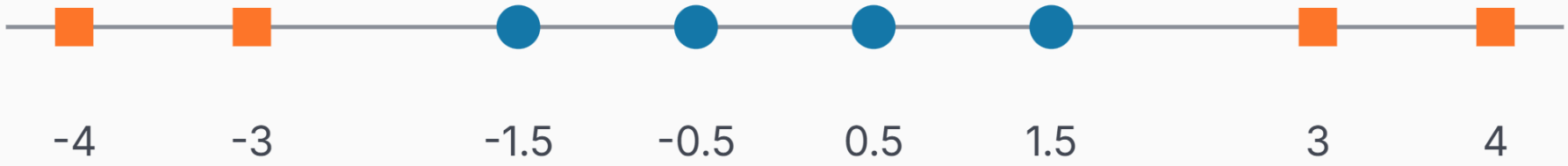
The same data with small C and large C



Small C: wide street, many support vectors (ringed)

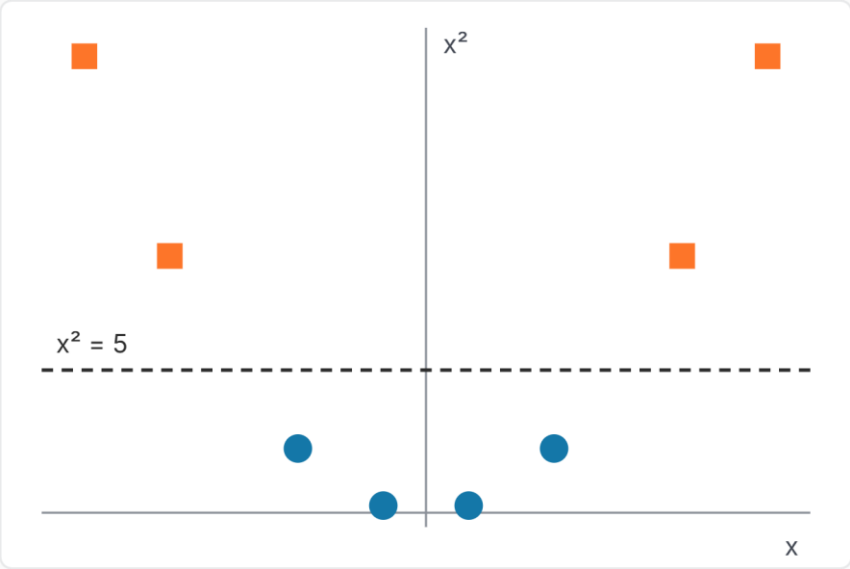
Large C: narrow street that tilts to the stray point

When no straight line can separate the data



On a number line, the boundary is a single cut point. The squares sit at both ends, so no single cut can put all circles on one side and all squares on the other.

Add a feature: map each x to (x, x²)



The squares rise high. The circles stay low.

A flat line at $x^2 = 5$ now separates them perfectly.

Back on the original number line, that line is two cut points: $x = \pm\sqrt{5}$, about ±2.24.

The idea: if the data cannot be separated where it is, move it to a higher dimension where it can.

Kernels you will use in scikit-learn

Kernel	What it does	Use it when
linear	No transformation	The data is roughly separable as it is
poly	Adds powers and products of features	You expect a smooth curved boundary
rbf (default)	Similarity falls off with distance	General-purpose curved or island-shaped boundaries

rbf = radial basis function. gamma (γ) sets how far each point's influence reaches; higher gamma may overfit, so tune it with C.

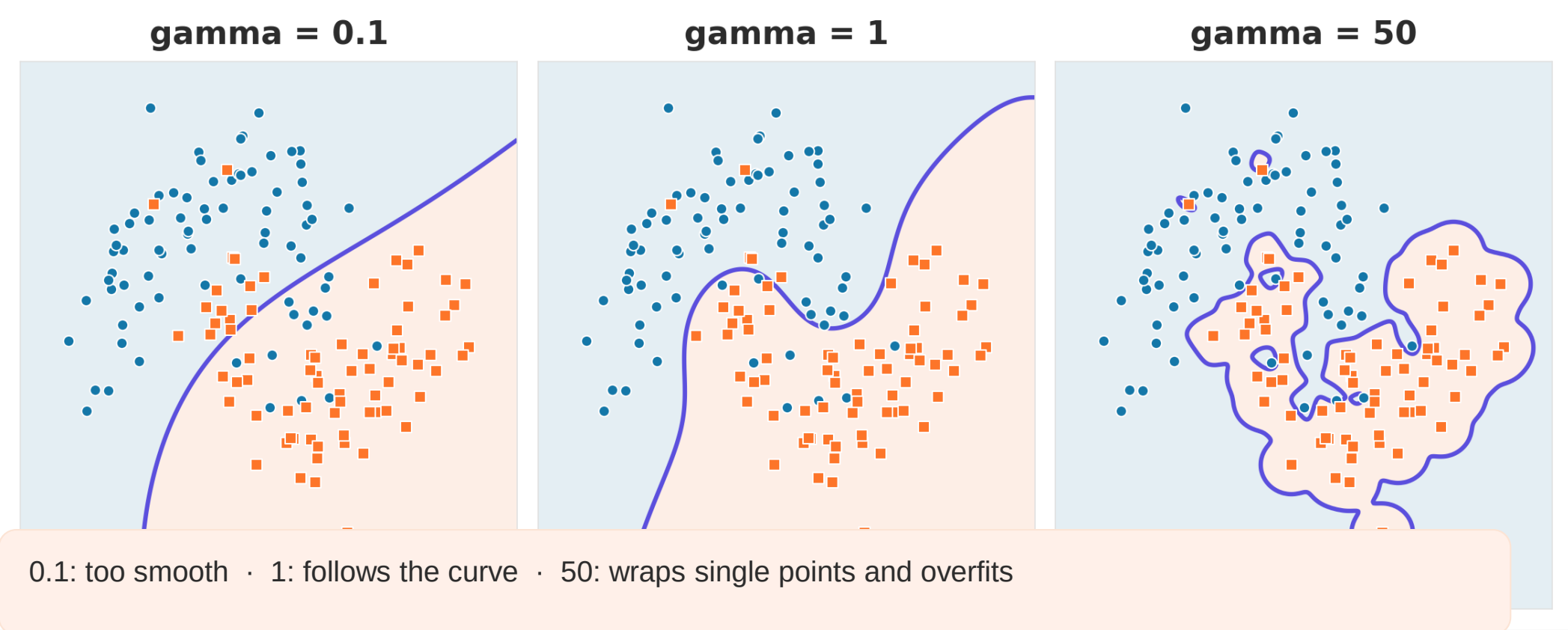
What gamma (γ) does: the RBF similarity

$K(x, z) = \exp(-\gamma \times \text{distance}^2)$ exp(n) means e^n, with e ≈ 2.718

Distance x to z	γ = 0.1	γ = 1	γ = 10
0.5	0.98	0.78	0.08
1	0.90	0.37	0.00005
3	0.41	0.0001	≈ 0

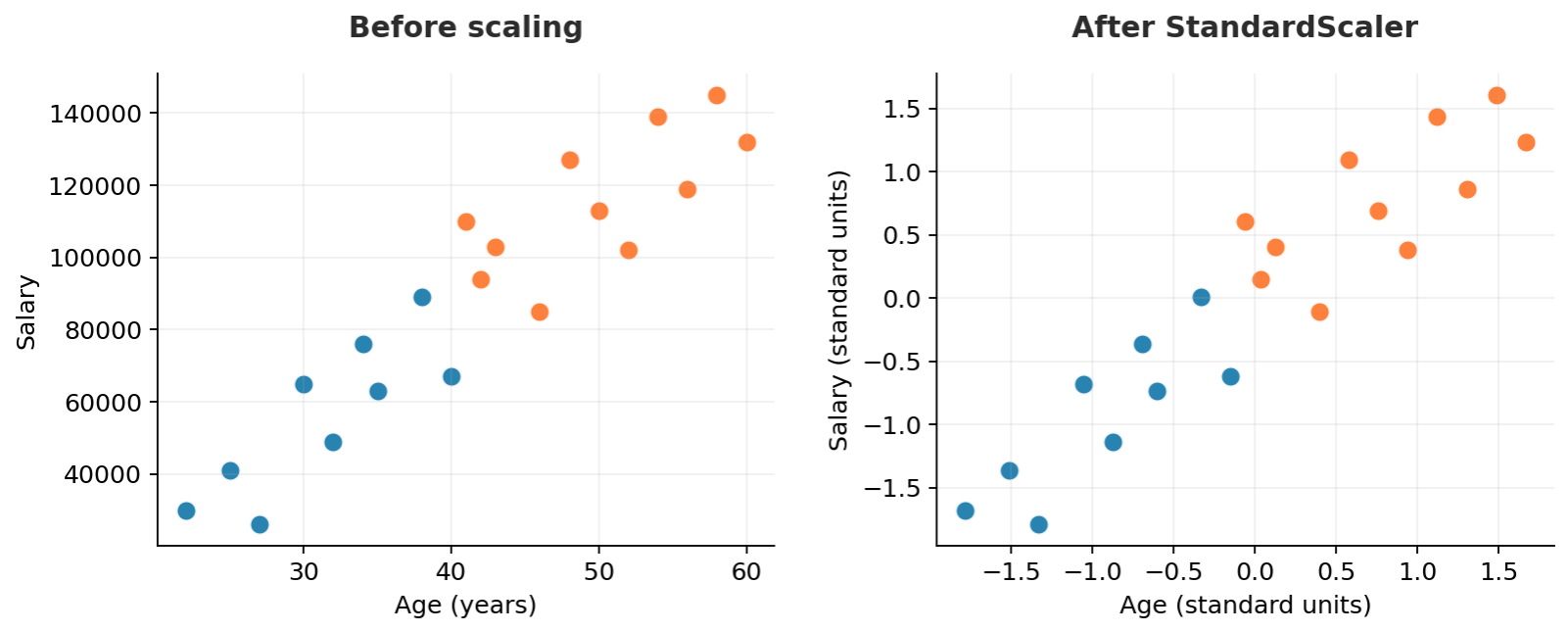
Example: distance 1, γ = 1 gives exp(−1) ≈ 0.37. K = 1 means the same point; K near 0 means no influence.

The same data with three gamma values



0.1: too smooth · 1: follows the curve · 50: wraps single points and overfits

Why SVM needs feature scaling



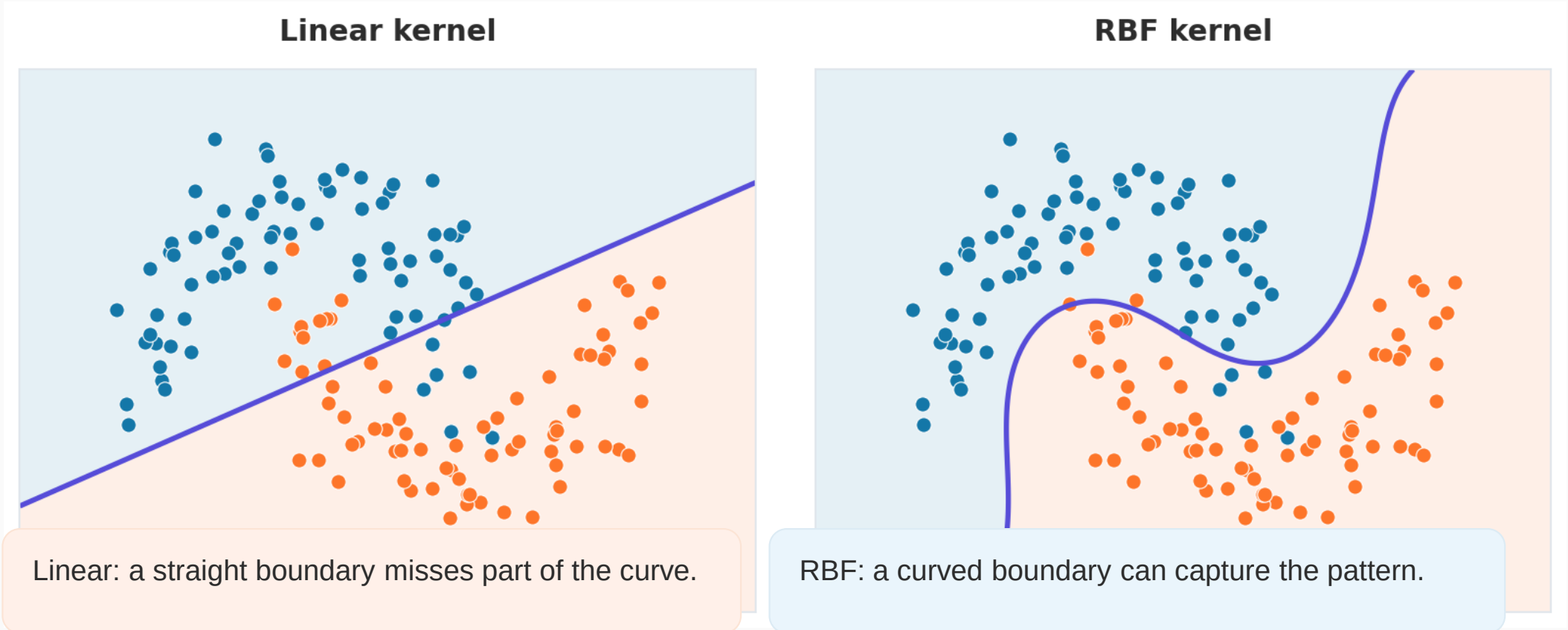
Think about the units

Age: 20–60
Salary: ₹20,000–₹150,000

Scaling puts them on comparable numerical scales.

StandardScaler: (value − mean) ÷ standard deviation. Put it in a Pipeline so it learns only from training data.

The same data: linear boundary vs RBF



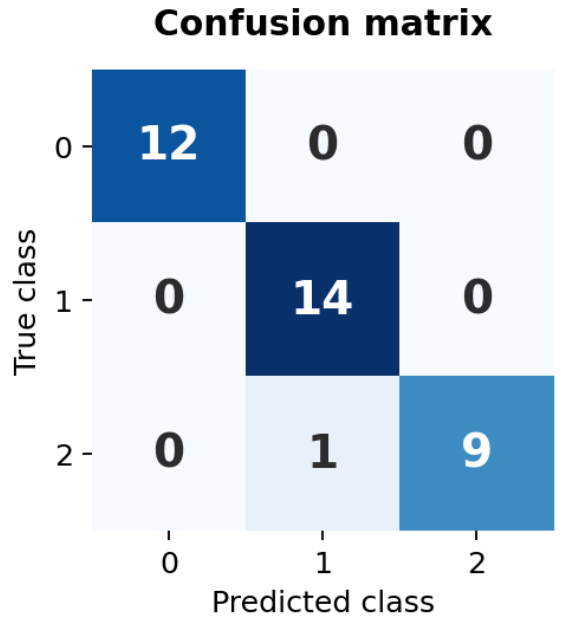
Linear: a straight boundary misses part of the curve.

RBF: a curved boundary can capture the pattern.

Train and evaluate SVM on the Wine dataset

Minimal Python workflow

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,  # 20% kept for testing
    stratify=y,           # same class mix in both parts
    random_state=42)      # same split on every run
model = make_pipeline(StandardScaler(), SVC())
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```



Accuracy on the 20% test data: 97.2%

Tune kernel, C and gamma with GridSearchCV

svc__C = C of the svc step · cv=5: 5 folds

```
params = [
    {"svc__kernel": ["linear"], "svc__C": [0.1, 1, 10]},
    {"svc__kernel": ["rbf"], "svc__C": [0.1, 1, 10],
     "svc__gamma": ["scale", 0.1]}
]
grid = GridSearchCV(
    make_pipeline(StandardScaler(), SVC()),
    params, cv=5, scoring="accuracy")
grid.fit(X_train, y_train)
print(grid.best_params_)
print(grid.score(X_test, y_test))
```

C Penalty for margin violations

gamma RBF influence range

5-fold CV best:

linear, C=0.1

CV 98.6% · Test 97.2%

Important: tune only using training data. Report the test score once after selecting the model.

Recap

SVM in four steps

01	**Maximum margin**	Of all separating hyperplanes, pick the one with the widest street.
02	**Support vectors**	Only the points on or inside the street decide where the boundary goes.
03	**Soft margin and C**	Allow some mistakes; C sets how much each one costs.
04	**Kernels**	Use a kernel for curved boundaries, without building new features.

Check your understanding

Answer in your own words

Question 1

Why might a wider margin help generalization?

Question 2

What happens to the boundary if you delete a point that is not a support vector?

Question 3

Which setting of C risks overfitting, and why?

Question 4

Why does SVM only need the support vectors to make a prediction?

OPTIONAL DEEP DIVE

How the kernel trick works

Dot products · dual coefficients · mathematical kernels

Use these four slides only if the class needs the mathematical explanation.

Dot product: alignment also depends on vector size

$a \cdot b = a_1b_1 + a_2b_2$ Multiply matching coordinates, then add.

Compare a = (3, 1) with	Dot product	Direction	Meaning
b = (3, 2)	9 + 2 = 11	Same way	Similar
c = (1, −3)	3 − 3 = 0	Right angle	Unrelated
d = (−3, −1)	−9 − 1 = −10	Opposite	Dissimilar

The sign indicates alignment; the value also depends on vector lengths. It is not a normalized similarity score.

Dot products are already inside SVM

Which side of the street?

$$\mathbf{w} \cdot \mathbf{x} + b$$

Positive: one class. Negative: the other. Zero: on the boundary. The margin edges are where it equals +1 and −1.

How SVM predicts

$$\text{sign}\left(\sum \alpha_i \, y_i \, (\mathbf{x}_i \cdot \mathbf{x}) + b \right)$$

α_i: learned weight of training point i (0 except for support vectors). y_i: its class. sign: +1 if positive, −1 if negative.

The margin is the **goal**. The dot product is the **tool** SVM uses to reach it and to make predictions.

Predictions depend only on the support vectors

$\alpha_i = 0$

For every point far from the street. It drops out of the sum.

$\alpha_i > 0$

Only for the support vectors, the points holding the street in place.

Example: train on 10,000 points and end up with 50 support vectors. Each prediction then needs 50 comparisons, not 10,000.

The kernel trick: dot products in a new feature space

Map each point: $x \rightarrow (x, x^2)$

Dot product in the new space: $(a, a^2) \cdot (b, b^2) = ab + a^2b^2$

Kernel: $K(a, b) = ab + (ab)^2$

Compute this value using a and b, without explicitly building the mapped coordinates.

Why it works

SVM's formula only uses dot products between points. Swap each one for $K(x_i, x)$.

The margin idea and C remain; the learned boundary and support vectors can change.